BENEMÉRITA UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Departamento de Ciencias Computacionales

Actividad de Aprendizaje 13. Tarea 7 Entrada-Salida

Investigación sobre manejo de archivos y directorios, índices y dispersión

2026-B

Sección D09

Sistemas Operativos

Código de la materia: IL366

Carrera: ICOM – Ingeniería en Computación

P R E S E N T A

ICOM – Velázquez Vazquéz Valeria – 224785718

PROFESOR(A)

Becerra Velázquez Violeta del Carmen

Guadalajara, Jalisco, Mx.

22 de Noviembre de 2026

ÍNDICE

Tabla de imágenes 3
Contenido 4
Introducción 4
1. Acciones en el manejo de archivos y directorios 5
2. Índices 7
3. Dispersión 8
4. Métodos de dispersión 9
5. Solución de colisiones 9
Conclusiones 11
Bibliografía 12

TABLA DE IMÁGENES

Figura 1. Índice denso sobre un archivo de datos desordenado.....7
Figura 2. Función de dispersión y colisión 8
Figura 3. Solución de colisiones: sondeo lineal y encadenamiento.....10

CONTENIDO

Introducción

Un archivo es una colección de información relacionada que se almacena en memoria secundaria y que el sistema operativo presenta al usuario como una unidad lógica; un directorio es la estructura que organiza los archivos y guarda información sobre ellos. Para que los programas puedan usarlos, el sistema operativo ofrece un conjunto de operaciones, normalmente disponibles como llamadas al sistema. Por otra parte, cuando los archivos crecen, localizar un registro recorriéndolo completo resulta demasiado lento, por lo que se recurre a técnicas de acceso directo como los índices y la dispersión (hashing).

Este documento lista las acciones posibles sobre archivos y directorios con su descripción, y resume los temas de índices, dispersión, métodos de dispersión y solución de colisiones. En las tablas se incluye, como referencia, la llamada al sistema equivalente en POSIX (Linux y UNIX).

1. Acciones en el manejo de archivos y directorios

Las acciones se agrupan según el objeto sobre el que operan. Un aspecto importante es que, para usar un archivo, primero se debe abrir: el sistema copia sus atributos a memoria y entrega al proceso un descriptor que se utiliza en las operaciones posteriores, hasta que el archivo se cierra.

1.1 Acciones sobre archivos

Acción	Llamada POSIX	Descripción
Crear	creat(), open()	Reserva espacio y una entrada en el directorio para un archivo nuevo, vacío y con los atributos iniciales (nombre, propietario, permisos).
Abrir	open()	Localiza el archivo, verifica los permisos y devuelve un descriptor para las operaciones siguientes, indicando el modo (lectura, escritura o ambos).
Cerrar	close()	Libera el descriptor y las estructuras asociadas; los datos pendientes se envían al disco.
Leer	read()	Transfiere bytes desde la posición actual del archivo a un búfer en memoria del proceso.
Escribir	write()	Transfiere datos de la memoria al archivo, a partir de la posición actual, sobrescribiendo el contenido existente.
Agregar	open() con O_APPEND	Escribe siempre al final del archivo, sin modificar lo que ya contiene.
Reposicionar	lseek()	Mueve el apuntador de posición a un punto dado sin leer ni escribir; permite el acceso directo (aleatorio).
Truncar	truncate(), ftruncate()	Reduce o amplía el tamaño del archivo; con longitud cero borra el contenido pero conserva los atributos.
Renombrar	rename()	Cambia el nombre (o la ubicación) del archivo modificando su entrada de directorio, sin copiar los datos.
Copiar	read() y write()	Crea un duplicado del contenido leyendo el origen y escribiendo en un archivo nuevo.
Eliminar	unlink()	Quita la entrada del directorio; los bloques se liberan cuando ya no hay enlaces ni procesos que lo tengan abierto.
Consultar atributos	stat(), fstat()	Obtiene información como tamaño, propietario, permisos, tipo y fechas de creación, acceso y modificación.
Modificar atributos	chmod(), chown(), utime()	Cambia los permisos, el propietario o las fechas del archivo.

Acción	Llamada POSIX	Descripción
Bloquear	flock(), fcntl()	Restringe el acceso concurrente al archivo o a una región para evitar inconsistencias entre procesos.

1.2 Acciones sobre directorios

Acción	Llamada POSIX	Descripción
Crear	mkdir()	Crea un directorio nuevo y vacío, que contiene únicamente las entradas “.” y “..”.
Eliminar	rmdir()	Borra un directorio; en la mayoría de los sistemas solo puede hacerse si está vacío.
Abrir	opendir()	Abre el directorio para poder recorrer sus entradas.
Leer (listar)	readdir()	Devuelve la siguiente entrada del directorio (nombre y número de nodo-i) cada vez que se invoca.
Cerrar	closedir()	Termina la lectura del directorio y libera sus recursos.
Renombrar	rename()	Cambia el nombre o la ubicación de un directorio dentro del sistema de archivos.
Cambiar de directorio	chdir()	Establece el directorio de trabajo del proceso, contra el cual se resuelven las rutas relativas.
Enlazar	link()	Crea un enlace duro: otro nombre para el mismo archivo, que aumenta su contador de referencias.
Enlace simbólico	symlink(), readlink()	Crea un archivo especial que contiene la ruta hacia otro archivo o directorio.
Buscar / recorrer	nftw(), find	Recorre el árbol de directorios para localizar archivos por nombre, tamaño, fecha u otro atributo.
Consultar atributos	stat()	Obtiene los atributos del directorio, como permisos, propietario y fechas.

En conjunto, estas acciones permiten crear, consultar, modificar y eliminar tanto los datos como la estructura de organización. El sistema operativo verifica en cada una los permisos del usuario, por lo que una acción puede fallar si no se tiene el derecho de lectura, escritura o ejecución correspondiente.

2. Índices

Un índice es una estructura auxiliar, separada del archivo de datos, que asocia el valor de una clave con la posición donde se encuentra el registro correspondiente. Funciona igual que el índice de un libro: en lugar de leer todas las páginas, se busca el tema en una lista ordenada y esta indica dónde ir. Cada entrada del índice contiene un par (clave, apuntador), donde el apuntador es el número de registro, la dirección del bloque o el desplazamiento dentro del archivo.

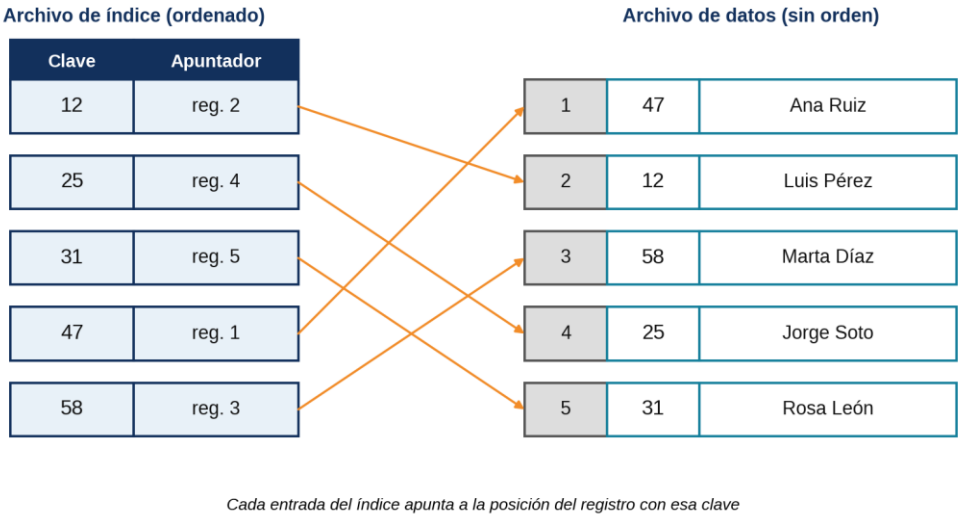


Figura 1. *Índice denso sobre un archivo de datos desordenado. Fuente: elaboración propia.*

Como el índice es mucho más pequeño que el archivo y está ordenado, permite buscar con una búsqueda binaria, o incluso mantenerse en memoria principal. Así, un archivo cuyos registros están en desorden puede consultarse rápidamente por la clave indexada. Los índices se clasifican de la siguiente forma:

- **Índice primario:** se construye sobre la clave por la que el archivo está ordenado físicamente.
- **Índice secundario:** se construye sobre un campo distinto al de ordenamiento; el archivo puede tener varios, uno por cada campo por el que se desee buscar.
- **Índice denso:** contiene una entrada por cada registro del archivo, como en la Figura 1.
- **Índice disperso (no denso):** contiene una entrada por bloque o grupo de registros; localiza el bloque adecuado y luego se busca dentro de él. Ocupa menos espacio, pero requiere que el archivo esté ordenado.
- **Índice multinivel:** cuando el índice es muy grande, se crea un índice del índice, y así sucesivamente. Los árboles B y B+ son la implementación más usada en sistemas de archivos y bases de datos.

La ventaja principal de los índices es la rapidez de consulta. Su desventaja es que ocupan espacio adicional y que cada inserción, borrado o modificación de la clave obliga a actualizarlos, lo que hace más lentas esas operaciones.

3. Dispersión

La dispersión (hashing) es una técnica de acceso directo que calcula la dirección de un registro a partir de su propia clave, sin necesidad de recorrer una estructura de búsqueda. Para ello se utiliza una función de dispersión h(k), que recibe la clave k y devuelve una posición dentro de una tabla de dispersión de m posiciones (0 a m − 1), como se ilustra en la Figura 2. Tanto para guardar como para buscar un registro se aplica la misma función, por lo que en el caso ideal se requiere un único acceso.

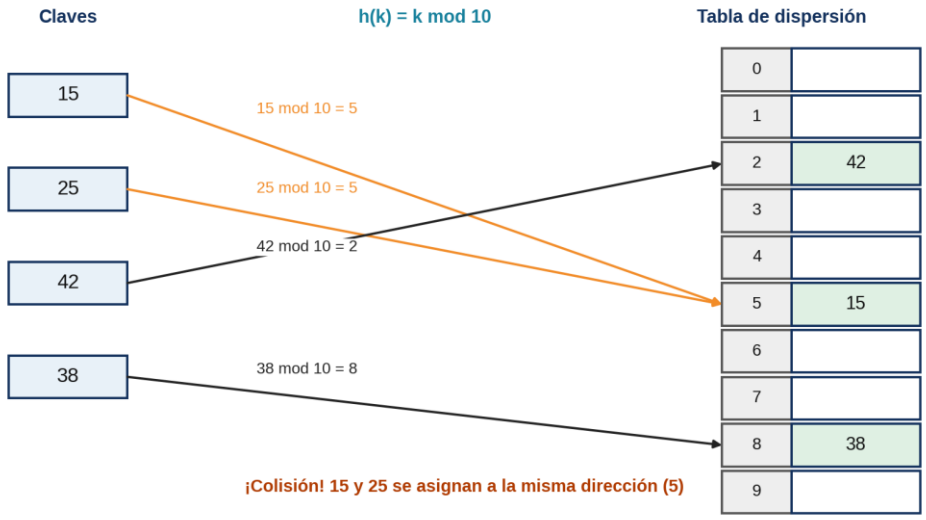


Figura 2. *Función de dispersión y colisión. Fuente: elaboración propia.*

Una buena función de dispersión debe ser fácil y rápida de calcular, distribuir las claves de manera uniforme en la tabla y aprovechar toda la clave para que claves parecidas no generen la misma dirección. Se define el **factor de carga** como $\alpha = n / m$, donde n es el número de registros almacenados; a medida que α se acerca a 1, la tabla se llena y aumenta la probabilidad de conflictos.

Sus ventajas son que el tiempo de acceso promedio es prácticamente constante y que no requiere mantener un índice ordenado. Sus desventajas son que no permite recorrer los registros en orden por la clave, que no soporta bien las búsquedas por rango y que exige manejar los casos en que dos claves distintas obtienen la misma dirección, llamados colisiones.

4. Métodos de dispersión

Existen varias formas de construir la función de dispersión. Los más comunes se resumen en la siguiente tabla, con un ejemplo para cada uno.

Método	Descripción	Ejemplo
Módulo (división)	h(k) = k mod m. Se toma el residuo de dividir la clave entre el tamaño de la tabla. Se recomienda que m sea un número primo para distribuir mejor las claves.	k = 1234, m = 97: 1234 mod 97 = 70, por lo tanto h(k) = 70.
Cuadrado medio	Se eleva la clave al cuadrado y se toman los dígitos centrales del resultado como dirección.	k = 453: 453^2 = 205209; dígitos centrales "52", por lo tanto h(k) = 52.
Plegamiento	La clave se divide en partes de igual longitud, que se suman; si la suma excede el tamaño de la dirección, se descarta el acarreo.	k = 123456789: 123 + 456 + 789 = 1368; se descarta el acarreo, por lo tanto h(k) = 368.
Truncamiento	Se eliminan algunos dígitos de la clave y se usan los restantes como dirección. Es simple, pero puede distribuir mal las claves.	k = 5837294: se toman los dígitos 1.º, 3.º y 5.º (5, 3, 2), por lo tanto h(k) = 532.
Multiplicación	$h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$, con $0 < A < 1$ (por ejemplo, $A \approx 0.618$). El resultado no depende tanto del valor de m.	k = 100, A = 0.618, m = 10: 61.8 mod 1 = 0.8, por lo tanto $h(k) = \lfloor 10 \cdot 0.8 \rfloor = 8$.
Conversión de base	La clave se interpreta como un número en otra base (por ejemplo, base 11) y se convierte a decimal, tomando después el residuo.	k = 372 en base 11: $3 \cdot 121 + 7 \cdot 11 + 2 = 442$; 442 mod 100 = 42.

Cuando las claves son alfanuméricas (por ejemplo, nombres), primero se convierten a un valor numérico sumando o combinando los códigos de sus caracteres, y luego se aplica alguno de los métodos anteriores. En la práctica, el método del módulo es el más utilizado por su sencillez.

5. Solución de colisiones

Una colisión ocurre cuando dos claves diferentes producen la misma dirección al aplicar la función de dispersión, como sucede en la Figura 2 con las claves 15 y 25, que corresponden a la dirección 5. Como es imposible evitarlas por completo, todo esquema de dispersión debe incluir un método para resolverlas. Se dividen en dos grandes familias.

5.1 Direccionamiento abierto (o cerrado)

Todos los registros se guardan dentro de la propia tabla. Si la posición asignada está ocupada, se busca otra libre siguiendo una secuencia de prueba (sondeo). La búsqueda sigue la misma secuencia hasta encontrar la clave o una posición vacía. Las variantes más comunes son:

- **Sondeo lineal:** se prueba la siguiente posición, (h(k) + i) mod m, con i = 1, 2, 3, ... Es simple, pero tiende a formar grupos de posiciones ocupadas (agrupamiento primario) que alargan las búsquedas.
- **Sondeo cuadrático:** los saltos crecen de forma cuadrática, $(h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$, lo que reduce el agrupamiento primario.
- **Doble dispersión:** se usa una segunda función para calcular el tamaño del salto, $(h_1(k) + i \cdot h_2(k)) \bmod m$. Distribuye mejor las claves que los anteriores.

En el ejemplo de la Figura 3a, con h(k) = k mod 10, las claves 15, 25, 35 y 42 se insertan en ese orden: el 15 ocupa la posición 5; el 25 también corresponde a la 5, que está ocupada, por lo que se coloca en la 6; el 35 encuentra ocupadas la 5 y la 6, y queda en la 7; y el 42 va directamente a la 2. Una desventaja es que al eliminar registros hay que marcar la posición como "borrada" y no como vacía, para no interrumpir las búsquedas.

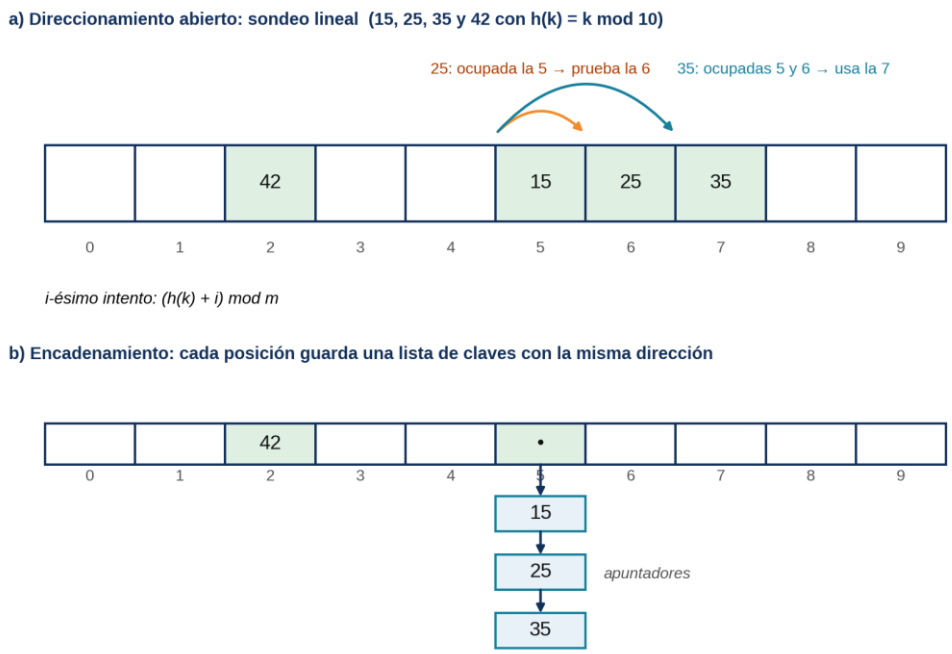


Figura 3. *Solución de colisiones: sondeo lineal y encadenamiento. Fuente: elaboración propia.*

5.2 Encadenamiento (direccionamiento cerrado)

Cada posición de la tabla guarda el inicio de una lista enlazada con todas las claves que se asignaron a esa dirección, como en la Figura 3b. Para insertar se calcula h(k) y se agrega la

clave a la lista, y para buscar se recorre únicamente la lista de esa posición. Su ventaja es que la tabla nunca se llena y que el borrado es sencillo; su desventaja es el espacio extra que ocupan los apuntadores y que las listas muy largas degradan el rendimiento.

5.3 Otras alternativas

- **Área de desbordamiento:** los registros que colisionan se guardan en una zona aparte del archivo, enlazados desde su dirección original.
- **Redispersión (rehashing):** cuando el factor de carga es alto, se crea una tabla más grande (por lo regular del doble de tamaño) y se vuelven a dispersar todas las claves con la nueva función.

CONCLUSIONES

Con esta investigación comprendí que el manejo de archivos y directorios se basa en un conjunto reducido de acciones básicas (crear, abrir, leer, escribir, cerrar, renombrar y eliminar, entre otras) que el sistema operativo expone mediante llamadas al sistema, y que sobre ellas se construyen todas las herramientas que uso a diario, como copiar o buscar archivos. También aprendí que cada acción se apoya en atributos y permisos, lo que muestra el papel del sistema operativo en la protección de la información.

Respecto a la organización de los datos, entendí que los índices y la dispersión son dos respuestas distintas al mismo problema: evitar leer todo el archivo. Los índices conservan el orden y permiten consultas por rango, pero ocupan espacio y deben mantenerse actualizados; la dispersión ofrece acceso casi inmediato, pero sacrifica el orden y necesita un método para resolver colisiones. Al elaborar los ejemplos noté que la elección de la función de dispersión y del tamaño de la tabla influye directamente en el número de colisiones, y que el sondeo lineal, aunque sencillo, genera agrupamientos que el encadenamiento y la doble dispersión evitan mejor.

BIBLIOGRAFÍA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Folk, M. J., Zoellick, B., y Riccardi, G. (1998). *File structures: An object-oriented approach with C++*. Addison-Wesley.

Kerrisk, M. (2010). *The Linux programming interface*. No Starch Press.

Knuth, D. E. (1998). *The art of computer programming, Vol. 3: Sorting and searching* (2.ª ed.). Addison-Wesley.

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

Silberschatz, A., Korth, H. F., y Sudarshan, S. (2020). *Database system concepts* (7.ª ed.). McGraw-Hill.

Tanenbaum, A. S., y Bos, H. (2015). *Modern operating systems* (4.ª ed.). Pearson.